



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное  
учреждение высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

---

ФАКУЛЬТЕТ \_\_\_\_\_ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ  
КАФЕДРА ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ (ИУ7)  
НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.04 Программная инженерия

**О т ч е т**  
**по лабораторной работе № 3**

Название: Организация памяти конвейерных суперскалярных  
электронных вычислительных машин

Дисциплина: Архитектура ЭВМ

Студент гр. ИУ7-52Б

\_\_\_\_\_  
(Подпись, дата)

М. А. Жаринов

(И.О. Фамилия)

Преподаватель

\_\_\_\_\_  
(Подпись, дата)

М.А. Гейне

(И.О. Фамилия)

2025 год

**Цель работы:** освоение принципов эффективного использования подсистемы памяти современных универсальных ЭВМ, обеспечивающей хранение и своевременную выдачу команд и данных в центральное процессорное устройство.

### **Эксперимент 1. Исследования расслоения динамической памяти**

Цель эксперимента: определение способа трансляции физического адреса, используемого при обращении к динамической памяти.

Исходные данные: размер линейки кэш-памяти верхнего уровня; объем физической памяти.

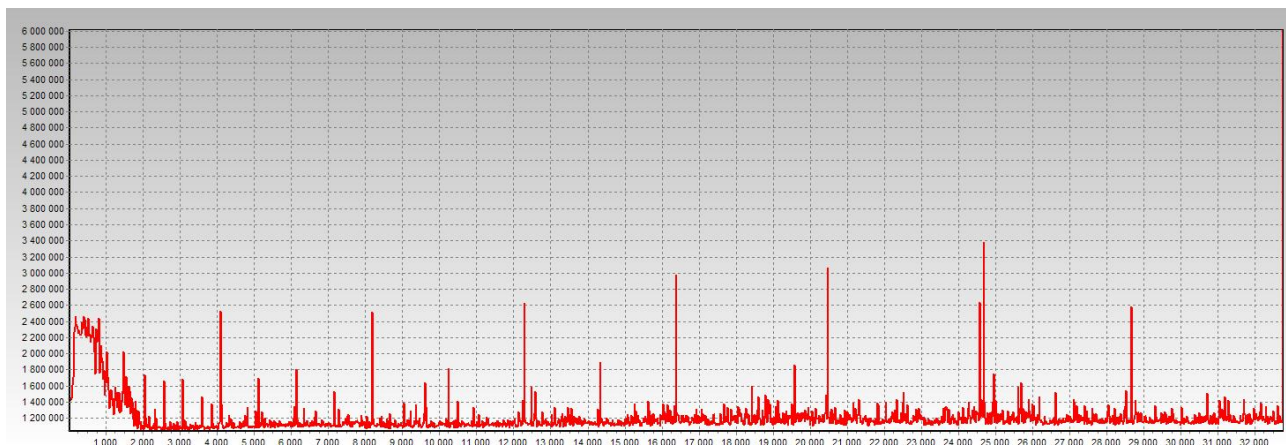
Для моего компьютера размер линейки  $\Pi = 64$ , объем памяти  $O = 16$  Гб.

Для определения способа трансляции физического адреса при формировании сигналов выборки банка, выборки строки и столбца запоминающего массива применяется процедура замера времени обращения к динамической памяти по последовательным адресам с изменяющимся шагом чтения. Для сравнения времен используется обращение к одинаковому количеству различных ячеек, отстоящих друг от друга на определенный шаг.

Для проведения эксперимента мною заданы параметры:

1. Максимальное расстояние между читаемыми блоками = 32,
2. Шаг увеличения расстояния между читаемыми 4-х байтовыми ячейками = 16,
3. Размер массива = 4.

После проведения эксперимента получены следующие результаты:



Минимальный шаг чтения динамической памяти, при котором происходит постоянное обращение к одному и тому же банку  $T1 = 256$ .

Количество банков памяти:  $B = T1/\Pi = 256 / 64 = 4$ .

Расстояние (в байтах) между началом двух последовательных страниц одного банка  $T2 = 16384$ .

Размер одной страницы:  $PC = T2/B = 16384 / 4 = 4096$ .

Количество страниц физической оперативной памяти:  $C = O/(PC*B*П) = (16 * 1024 * 1024 * 1024) / (4096 * 4 * 64) = 16384$ .

Выводы:

1.  $B = 4$ ;
2.  $PC = 4096$ ;
3.  $C = 16384$ ;
4. Обращение к последовательно расположенным данным требует различного времени;
5. Для создания эффективных программ необходимо учитывать расслоение памяти, характеризуемое способом трансляции физического адреса.

## Эксперимент 2. Сравнение эффективности ссылочных и векторных структур

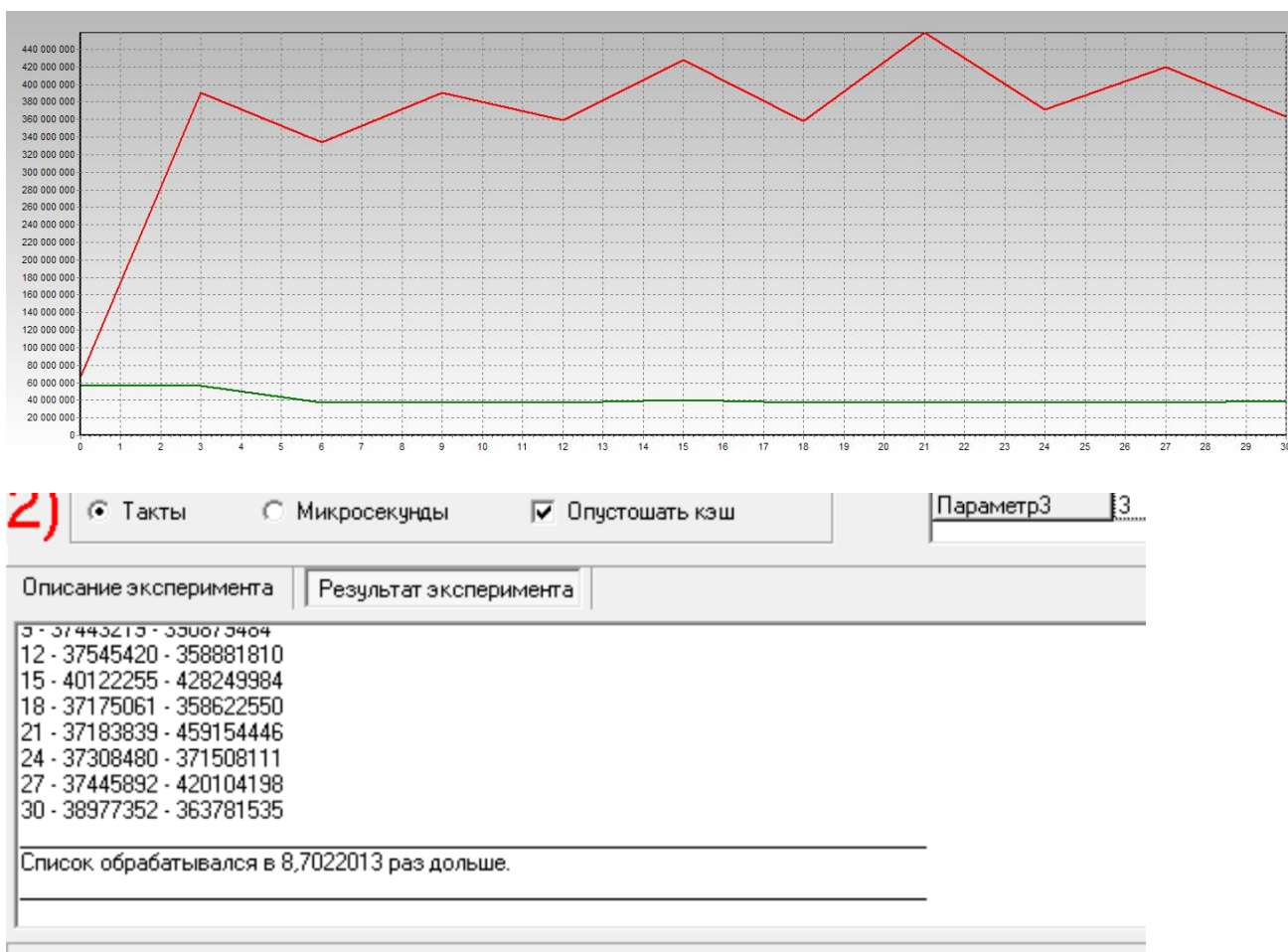
Цель эксперимента: оценка влияния зависимости команд по данным на эффективность вычислений.

Для сравнения эффективности векторных и списковых структур в эксперименте применяется профилировка кода двух алгоритмов поиска минимального значения. Первый алгоритм использует для хранения данных список, в то время как во втором применяется массив.

Для проведения эксперимента мною заданы параметры:

1. Количество элементов в списке = 8,
2. Максимальная фрагментации списка = 30,
3. Шаг увеличения фрагментации = 3.

После проведения эксперимента получены следующие результаты:



Таким образом, отношение времени работы алгоритма, использующего зависимые данные, ко времени обработки аналогичного алгоритма обработки независимых данных равно 8.7.

Выводы:

Обработка ссылочных структур процессорами с длинными конвейерами команд приводит к заметному увеличению времени работы алгоритмов: адрес загружаемого операнда становится известным только после обработки предыдущей команды. В противоположность этому, обработка векторных структур, таких как массивы, позволяет эффективно использовать аппаратные возможности ЭВМ.

### Эксперимент 3. Исследование эффективности программной предвыборки

Цель эксперимента: выявление способов ускорения вычислений благодаря применению предвыборки данных

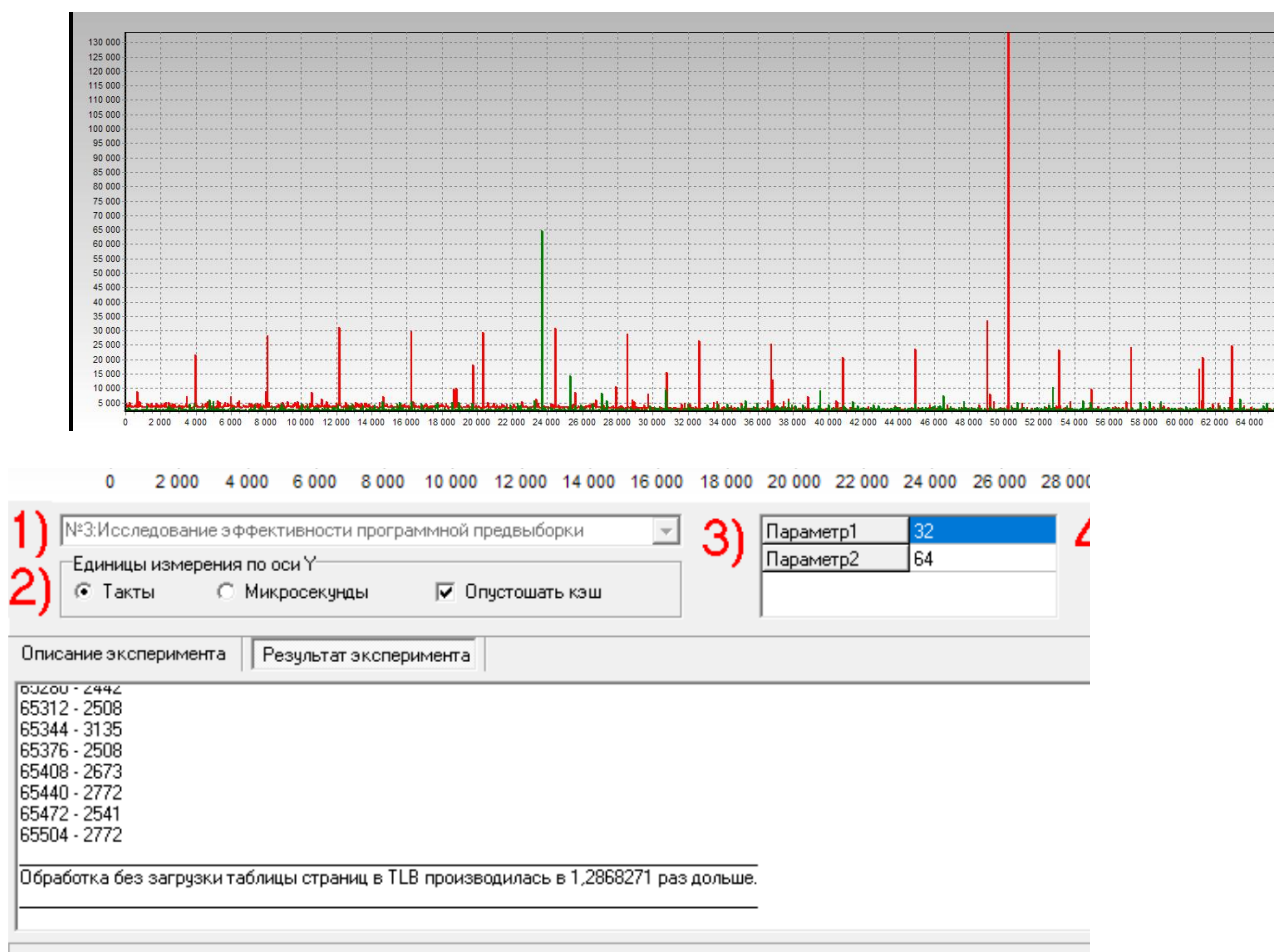
Исходные данные: степень ассоциативности и размер TLB данных.

Эксперимент основан на замере времени двух вариантов подпрограмм последовательного чтения страниц оперативной памяти. В первом варианте выполняется последовательное чтение без дополнительной оптимизации, что приводит к дополнительным двойным обращениям. Во втором варианте перед циклом чтения страниц используется дополнительный цикл предвыборки, обеспечивающий своевременную загрузку информации в TLB данных.

Для проведения эксперимента мною заданы параметры:

1. Шаг увеличения расстояния между читаемыми данными = 32,
2. Размер массива = 64.

После проведения эксперимента получены следующие результаты:



Таким образом, отношение времени последовательной обработки блока данных ко времени обработки блока с применением предвыборки равно 1.29, количество тактов первого обращения к странице данных в среднем равно 27000.

Выводы:

При обработке больших массивов информации целесообразно производить предвыборку, что позволяет избежать замедления работы в 1.3 раза.

## Эксперимент 4. Исследование способов эффективного чтения оперативной памяти

Цель эксперимента: исследование возможности ускорения вычислений благодаря использованию структур данных, оптимизирующих механизм чтения оперативной памяти

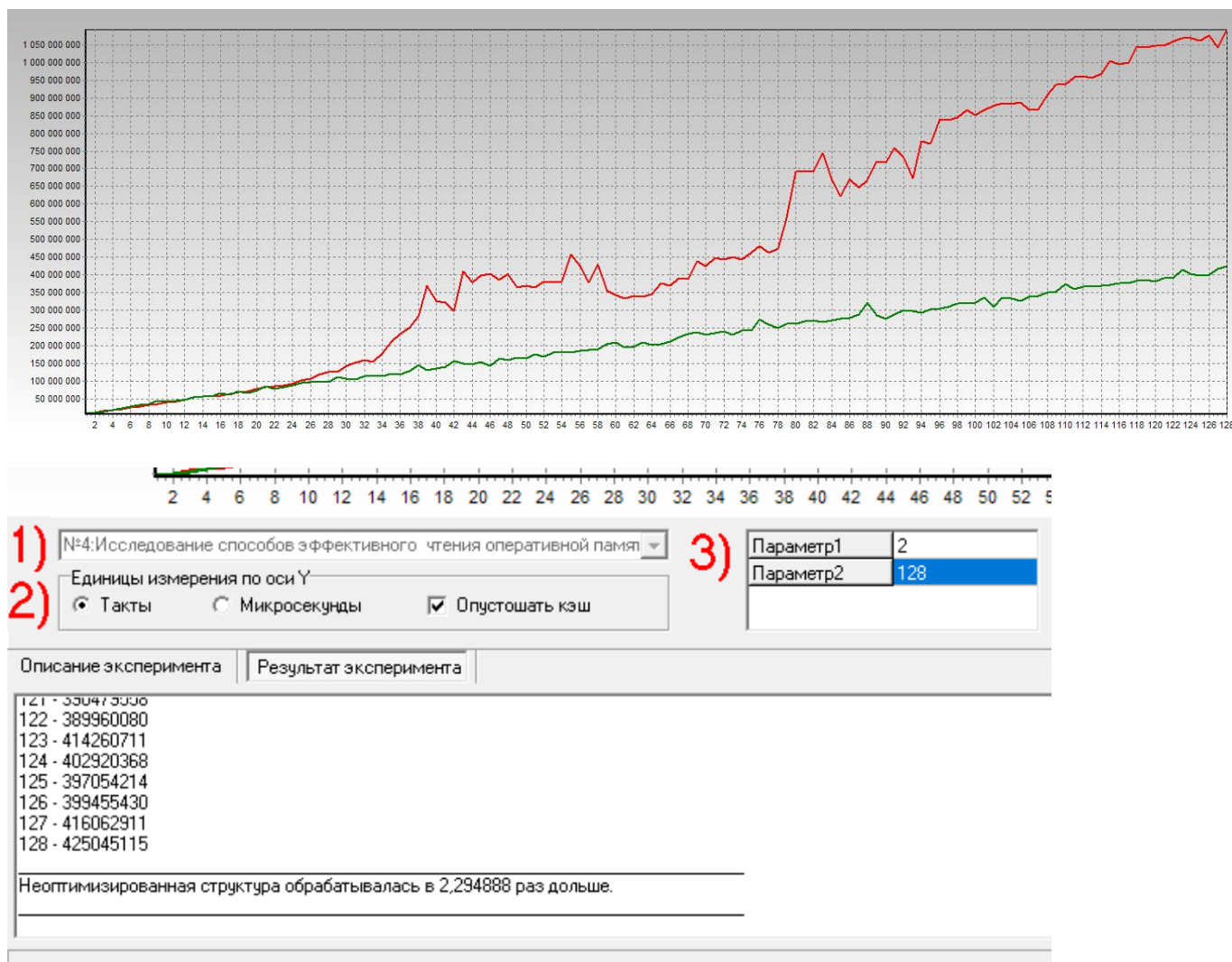
Исходные данные: Адресное расстояние между банками памяти, размер буфера чтения.

Неоптимизированный вариант структуры данных представляет собой несколько массивов в оперативной памяти, в то время как оптимизированная структура состоит из чередующихся данных каждого массива.

Для проведения эксперимента мною заданы параметры:

1. Размер массива = 2,
2. Количество потоков данных = 128.

После проведения эксперимента получены следующие результаты:





Таким образом, отношение времени обработки блока памяти неоптимизированной структуры ко времени обработки блока структуры, обеспечивающей эффективную загрузку и параллельную обработку данных, равно 2.29.

Выводы:

Для создания структур данных, оптимизирующих их обработку современными процессорами, требуется максимально исключить несвоевременную передачу данных, т.е. передавать в каждом пакете только востребованную для вычислений информацию, что позволяет избежать замедления работы в 2.3 раза.

## Эксперимент 5. Исследование конфликтов в кэш-памяти

Цель эксперимента: исследование влияния конфликтов кэш-памяти на эффективность вычислений.

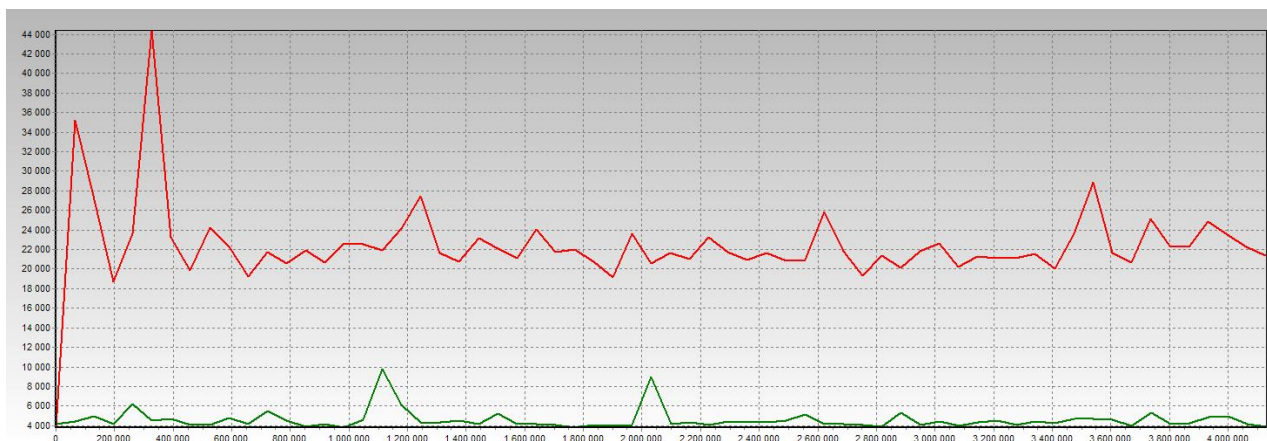
Исходные данные: Размер банка кэш-памяти данных первого и второго уровня, степень ассоциативности кэш-памяти первого и второго уровня, размер линейки кэш-памяти первого и второго уровня.

Для определения степени влияния конфликтов в кэш-памяти на эффективность вычислений используется профилировка двух процедур чтения и обработки данных. Первая процедура построена таким образом, что чтение данных выполняется с шагом, кратным размеру банка. Это порождает постоянные конфликты в кэш-памяти. Вторая процедура оптимизирует размещение данных в кэш с помощью задания смещения востребованных данных на некоторый шаг, достаточный для выбора другого набора. Этот шаг соответствует размеру линейки.

Для проведения эксперимента мною заданы параметры:

1. Размер банка кэш-памяти = 64,
2. Размер линейки кэш-памяти = 64,
3. Количество читаемых линеек = 64.

После проведения эксперимента получены следующие результаты:



| Описание эксперимента                                             | Результат эксперимента |
|-------------------------------------------------------------------|------------------------|
| 3671000 - 3360                                                    |                        |
| 3737376 - 5313                                                    |                        |
| 3802944 - 4158                                                    |                        |
| 3868512 - 4257                                                    |                        |
| 3934080 - 4851                                                    |                        |
| 3999648 - 4950                                                    |                        |
| 4065216 - 4191                                                    |                        |
| 4130784 - 3927                                                    |                        |
| Чтение с конфликтами банков производилось в 4,8978234 раз дольше. |                        |
|                                                                   |                        |
|                                                                   |                        |

Таким образом, отношение времени обработки массива с конфликтами в кэш-памяти ко времени обработки массива без конфликтов, равно 4.9.

Выводы:

При разработке программ стоит избегать конфликтов в кэш-памяти, что позволяет избежать замедления работы в 4.9 раза.

## Эксперимент 6. Сравнение алгоритмов сортировки

Цель эксперимента: исследование способов эффективного использования памяти и выявление наиболее эффективных алгоритмов сортировки, применимых в вычислительных системах.

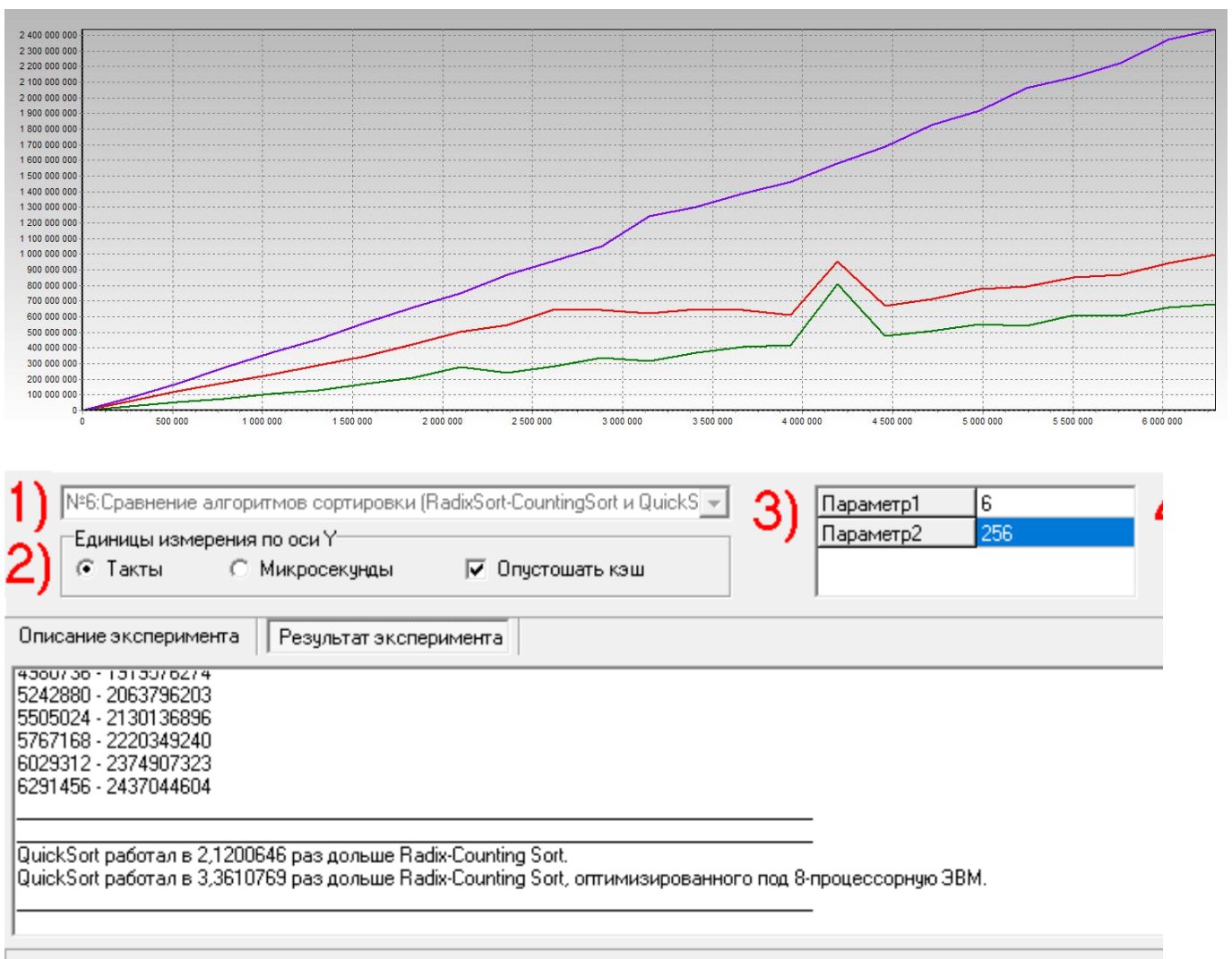
Исходные данные: количество процессоров вычислительной системы, размер пакета, количество элементов в массиве, разрядность элементов массива.

Эксперимент основан на замере времени трех вариантов алгоритмов сортировки (Quick Sort, Radix-Counting Sort, Оптимизированный Radix-Counting Sort).

Для проведения эксперимента мною заданы параметры:

1. Количество 64-х разрядных элементов массивов = 6,
2. Шаг увеличения размера массива = 256.

После проведения эксперимента получены следующие результаты:



Таким образом, отношение времени сортировки массива алгоритмом QuickSort ко времени сортировки алгоритмом Radix-Counting Sort и ко времени сортировки Radix-Counting Sort, оптимизированной под 8-процессорную вычислительную систему, равно 2.12 и 3.36 соответственно.

Выводы:

Для выполнения сортировки на многопроцессорных ЭВМ требуется использовать оптимизированную стратегию Radix-Counting сортировки, учитывающую количество процессоров, что позволяет избежать замедления работы в 2 и более раз.

## Информация об идентификации процессора

| Методические указания | Идентификация процессора                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | <p>19 - reserved<br/> MMX instructions<br/> FXSAVE/FXRSTOR<br/> 25 - reserved<br/> 26 - reserved<br/> 28 - reserved<br/> Generation: 15 Model: 4<br/> Extended feature flags 2fd3fbff:<br/> Floating Point Unit<br/> Virtual Mode Extensions<br/> Debugging Extensions<br/> Page Size Extensions<br/> Time Stamp Counter (with RDTSC and CR4 disable bit)<br/> Model Specific Registers with RDMSR &amp; WRMSR<br/> PAE - Page Address Extensions<br/> Machine Check Exception<br/> COMPXCHG8B Instruction<br/> APIC<br/> SYSCALL/SYSRET or SYSENTER/SYSEXIT instructions<br/> MTRR - Memory Type Range Registers<br/> Global paging extension<br/> Machine Check Architecture<br/> Conditional Move Instruction<br/> PAT - Page Attribute Table<br/> PSE-36 - Page Size Extensions<br/> 20 - reserved<br/> AMD MMX Instruction Extensions<br/> MMX instructions<br/> FXSAVE/FXRSTOR<br/> 25 - reserved<br/> 26 - reserved<br/> 27 - reserved<br/> 29 - reserved</p> <p>Processor name string: AMD Ryzen 5 7535HS with Radeon Graphics<br/> L1 Cache Information:<br/> 2/4-MB Pages:<br/> Data TLB: associativity 255-way #entries 64<br/> Instruction TLB: associativity 255-way #entries 64<br/> 4-KB Pages:<br/> Data TLB: associativity 255-way #entries 64<br/> Instruction TLB: associativity 255-way #entries 64<br/> L1 Data cache:<br/> size 32 KB associativity 8-way lines per tag 1 line size 64<br/> L1 Instruction cache:<br/> size 32 KB associativity 8-way lines per tag 1 line size 64</p> <p>L2 Cache Information:<br/> 2/4-MB Pages:<br/> Data TLB: associativity 16-way #entries 0<br/> Instruction TLB: associativity 2-way #entries 0<br/> 4-KB Pages:<br/> Data TLB: associativity 16-way #entries 0<br/> Instruction TLB: associativity 2-way #entries 0<br/> size 2 KB associativity L2 off lines per tag 97 line size 64</p> <p>Advanced Power Management Feature Flags<br/> Has temperature sensing diode<br/> Maximum linear address: 48; maximum phys address 48</p> |

### **Вывод**

В процессе выполнения лабораторной работы мною были освоены принципы эффективного использования подсистемы памяти современных универсальных ЭВМ, обеспечивающей хранение и своевременную выдачу команд и данных в центральное процессорное устройство.

## Контрольные вопросы

### 1. Назовите причины расслоения оперативной памяти.

Причиной расслоения является конструктивная неоднородность оперативной памяти. Из-за нее обращение к последовательно расположенным данным требует различного времени. Расслоение (разделение на банки) необходимо учитывать для создания эффективных программ, чтобы обеспечить параллельный доступ или скрыть задержки при закрытии/открытии страниц памяти.

### 2. Как в современных процессорах реализована аппаратная предвыборка.

Процессор имеет специализированные блоки, которые анализируют поток обращений к памяти. Если выявляется закономерность (например, последовательное чтение массива), механизм аппаратной предвыборки автоматически загружает следующие блоки данных из оперативной памяти в кэш до того, как программа явно их запросит.

### 3. Какая информация храниться в TLB.

В буфере TLB хранится информация о использованных ранее страницах памяти.

### 4. Какой тип ассоциативной памяти используется в кэш-памяти второго уровня современных ЭВМ и почему.

В кэш-памяти второго уровня современных ЭВМ используется наборно-ассоциативная организация, так как она представляет собой наиболее эффективный компромисс между сложностью полной ассоциативности и ограничениями прямого отображения. Такой тип памяти разбивает кэш на группы (наборы) из нескольких линеек, что позволяет данным с одинаковым индексом (младшей частью адреса) размещаться в любой свободной линейке внутри своего набора. Это существенно снижает количество конфликтов и «промахов» при обращении к памяти (когда разные данные претендуют на одно место), предотвращая постоянное вытеснение активных данных при работе с массивами, но при этом не требует дорогостоящей аппаратной реализации, как полностью ассоциативная память.

### 5. Приведите пример программной предвыборки.

```
// БЕЗ ПРЕДВЫБОРКИ
for (int a = 0; a < Param_[2]; a += Param_[1])
    x += *(int *) (int(p) + a);

// С ПРЕДВЫБОРКОЙ
for (int i = 0; i < Param_[2]; i += 4*K) x += *(int
*) ((int)p + i);
for (int a = 0; a < Param_[2]; a += Param_[1])
    x += *(int *) (int(p) + a);
```